PeerIslands

# GitHub Spec-Kit

## Complete Command Reference

Usage & Explanation of Every /speckit Command

For use with Claude Code, GitHub Copilot, Gemini CLI, and 20+ AI agents

github.com/github/spec-kit · v0.7.0 · 88k+ Stars

**PeerIslands · MongoDB AI Conference 2026**

# Overview: The SDD Workflow

Spec-Driven Development follows a structured 5-phase workflow. Each phase has a dedicated command. The key principle: specifications are written BEFORE code, not after. Code implements the spec — if they disagree, fix the code, not the spec.

**The commands are designed to be run in this order:**

| # | Command | Phase | What It Defines |
|---|---|---|---|
| 1 | `/speckit.constitution` | Foundation | Principles, standards, non-negotiable rules |
| 2 | `/speckit.specify` | Specification | WHAT and WHY — business requirements, user stories |
| 3 | `/speckit.clarify` | Clarification | Resolve ambiguities through targeted Q&A |
| 4 | `/speckit.plan` | Planning | HOW — tech stack, architecture, data model, APIs |
| 5 | `/speckit.tasks` | Task Breakdown | Dependency-ordered, parallelizable task list |
| 6 | `/speckit.analyze` | Consistency | Cross-artifact validation (pre & post build) |
| 7 | `/speckit.implement` | Implementation | Execute tasks → build the application |
| 8 | `/speckit.checklist` | Validation | Acceptance criteria pass/fail verification |
| 9 | `/speckit.taskstoissues` | Tracking | Convert tasks to GitHub Issues |

# /speckit.constitution

**Phase 0 — Foundation**

## Purpose

Create or update the project's governing principles, coding standards, and non-negotiable rules that all subsequent specs, plans, and code must adhere to.

## When to Use

Run FIRST, before any other command. Defines the rulebook for the entire project.

## What It Does

- Creates .specify/memory/constitution.md with your project's foundational guidelines
- Establishes coding standards (naming conventions, architecture patterns, test coverage)
- Defines non-negotiable principles (e.g., 'spec is source of truth', 'no code without spec')
- Sets design standards (color palette, typography, component library)
- The AI agent references this file during ALL subsequent phases

## Output File

`.specify/memory/constitution.md`

## Example Usage

```
/speckit.constitution Create principles for SmartInvoice — an AI-powered invoice manager for
small businesses in India. Focus on: spec is source of truth, spec before code, human-reviewable
specs with ASCII wireframes, layered hierarchy (business → platform → features), minimal
specification, continuous validation. Standards: 18% GST tax, Indian Rupee currency, DD/MM/YYYY
dates, sequential invoice numbering.
```

## Tips & Best Practices

- Keep it focused — overly strict constitutions (e.g., 80% test coverage, performance budgets) make everything downstream complex
- Include tech-agnostic principles (not framework-specific rules)
- You can update the constitution later by running the command again

# /speckit.specify

**Phase 1 — Specification**

## Purpose

Transform a business idea or vague requirements into a structured, detailed product specification with user stories, requirements, and acceptance criteria.

## When to Use

Run AFTER constitution. Describe WHAT you want to build and WHY. Do NOT include tech stack here.

## What It Does

- Creates a new git branch (e.g., 001-smartinvoice)
- Generates specs/001-smartinvoice/spec.md with structured requirements
- Extracts user stories with numbered acceptance criteria
- Identifies ambiguities and marks them with [NEEDS CLARIFICATION] tags
- Creates a Review & Acceptance Checklist at the end of the spec

## Output File

`.specify/specs/<feature>/spec.md`

## Example Usage

```
/speckit.specify Build SmartInvoice — an invoice manager for Indian small businesses with 3 use
cases: (1) Invoice Management — create invoices with client details, dynamic line items, auto-
calc 18% GST, sequential INV-XXXX numbering, lifecycle draft→pending→paid/overdue. (2) Dashboard
— real-time stats showing total revenue, pending, overdue amounts, monthly revenue chart, recent
invoices. (3) AI Insights — client payment scoring by reliability, risk classification
Low/Medium/High, 3-month revenue forecast, top clients ranking.
```

## Tips & Best Practices

- Be explicit about WHAT and WHY — not HOW (save tech stack for /speckit.plan)
- Don't treat the first output as final — iterate with /speckit.clarify
- Include business context: who are the users, what problem does this solve
- The more specific you are about requirements, the better the generated spec

# /speckit.clarify

**Phase 1.5 — Clarification (Optional but Recommended)**

## Purpose

Identify and resolve ambiguities in the specification through targeted questions. Catches gaps that would become bugs if left unresolved.

## When to Use

Run AFTER /speckit.specify and BEFORE /speckit.plan. Can also run after /speckit.plan for technical clarification.

## What It Does

- Reads the current spec.md and identifies underspecified areas
- Asks 5+ targeted questions with suggested answers (multiple choice)
- Records your answers in a Clarifications section of the spec
- Updates the spec.md with the clarified details
- Can be run multiple times for iterative refinement (e.g., BA review, stakeholder review)

## Output File

`Updates existing spec.md (adds Clarifications section)`

## Example Usage

```
/speckit.clarify Focus on edge cases and error handling requirements.
```

## Example Clarification Q&A

- *Q: What tax structure applies? → A: 18% GST flat rate*
- *Q: What happens when a client has overdue invoices? → A: Show warning badge, still allow new invoices*
- *Q: Should invoice numbers reset annually? → A: No, continuous sequential*
- *Q: What email validation is needed? → A: Standard regex, required field*
- *Q: What's the minimum line items per invoice? → A: At least 1 required*

## Tips & Best Practices

- Run this BEFORE /speckit.plan to reduce rework downstream
- You can provide focus areas: 'Focus on security and performance requirements'
- Can be run multiple times to simulate BA review → stakeholder review iterations
- If skipping intentionally (spike/prototype), state it explicitly so the agent doesn't block

# /speckit.plan

**Phase 2 — Planning**

## Purpose

Create a technical implementation plan with your chosen tech stack, architecture decisions, data models, API contracts, and component structure.

## When to Use

Run AFTER specification is finalized (specify + clarify done). This is where you define the tech stack.

## What It Does

- Generates plan.md with architecture, data model, and API design
- Creates research.md with tech stack analysis and library comparisons
- Creates data-model.md with database schema definitions
- Creates contracts/ directory with API specifications (REST/WebSocket)
- Creates quickstart.md with key validation scenarios
- Validates tech stack choices against the constitution

## Output File

`.specify/specs/<feature>/plan.md (+ research.md, data-model.md, contracts/)`

## Example Usage

```
/speckit.plan React with hooks for frontend. Dark theme with MongoDB green (#00ED64) accent. In-memory data layer simulating MongoDB. Business logic: calcLineTotal = qty × price, calcSubtotal = Σ(line totals), calcTax = subtotal × 0.18, calcGrandTotal = subtotal + tax. Indian Rupee ₹ formatting with toLocaleString("en-IN"). 8 REST API endpoints. Seed data: 8 invoices across 4 clients.
```

## Tips & Best Practices

- This is the ONLY place to define tech stack — deliberately separated from /speckit.specify
- If the client later says 'switch from React to Vue', only the plan changes — the spec stays untouched
- Ask the AI to research rapidly-changing frameworks before finalizing
- Review the plan for over-engineering — AI agents tend to add unnecessary complexity

# /speckit.tasks

**Phase 3 — Task Breakdown**

## Purpose

Generate a dependency-ordered, actionable task list from the implementation plan. Tasks are granular enough for AI to execute precisely.

## When to Use

Run AFTER /speckit.plan is finalized. Creates the execution roadmap.

## What It Does

- Generates tasks.md with ordered tasks grouped by user story/phase
- Manages dependencies between tasks (models before services, services before endpoints)
- Marks parallelizable tasks with [P] for concurrent execution
- Includes exact file paths for each task
- Adds checkpoint validations after each phase
- If constitution requires tests, test tasks are ordered before implementation

## Output File

`.specify/specs/<feature>/tasks.md`

## Example Usage

```
/speckit.tasks
```

## Tips & Best Practices

- This command takes NO arguments — it reads the plan.md automatically
- Review the task order for logical dependencies
- Tasks are deliberately granular — 'write task for the model to follow' is the goal
- For large projects, implement in phases (Phase 1: core, Phase 2: features, Phase 3: polish)

# /speckit.analyze

**Phase 3.5 / Phase 5 — Consistency Check (Optional but Recommended)**

## Purpose

Cross-check all spec artifacts (spec.md, plan.md, tasks.md) for consistency, gaps, and drift between specification and implementation.

## When to Use

Run AFTER /speckit.tasks (pre-build check) and/or AFTER /speckit.implement (post-build validation).

## What It Does

- Cross-references spec.md, plan.md, and tasks.md for consistency
- Detects conflicting API routes across different specs
- Validates that all requirements have corresponding tasks
- Checks enum consistency (e.g., status values match everywhere)
- Reports drift between what was specified and what was built (post-build)
- Flags missing acceptance criteria coverage

## Output File

`Console output (analysis report)`

## Example Usage

```
/speckit.analyze
```

## Tips & Best Practices

- Run before /speckit.implement as a pre-build gate — catches issues cheaply
- Run after /speckit.implement to verify spec-to-code traceability
- If analyze finds drift, fix the CODE — not the spec (Principle #1)

# /speckit.implement

**Phase 4 — Implementation**

## Purpose

Execute all tasks from tasks.md to build the complete application. The AI reads the spec, plan, and tasks, then generates code that implements each task in order.

## When to Use

Run AFTER tasks are generated and reviewed. This is the build step.

## What It Does

- Validates prerequisites: constitution, spec, plan, and tasks must all exist
- Parses tasks.md and executes tasks in dependency order
- Respects [P] parallel markers for concurrent execution
- Generates actual code files in your project directory
- Runs CLI commands as needed (npm install, migrations, etc.)
- Provides progress updates and handles errors
- Each generated line of code traces back to a spec requirement

## Output File

`Your complete application codebase`

## Example Usage

```
/speckit.implement
```

## Tips & Best Practices

- The AI will execute local CLI commands (npm, dotnet, pip, etc.) — ensure tools are installed
- For large projects, implement in phases: '/speckit.implement Phase 1 only'
- Test the application and paste runtime errors back to the AI for resolution
- Browser console errors won't be visible in CLI logs — check manually
- This command takes NO arguments — it reads tasks.md automatically

# /speckit.checklist

**Phase 5 — Validation**

## Purpose

Generate a custom quality checklist that validates requirements completeness, clarity, and consistency — like 'unit tests for English'.

## When to Use

Run AFTER implementation to verify acceptance criteria, or DURING specification to audit spec quality.

## What It Does

- Reads the spec and generates domain-specific validation checkpoints
- Creates pass/fail criteria for each requirement
- Covers areas like UX, security, accessibility, performance
- Asks clarifying questions to ensure checklist accuracy
- Can be used to walk through the running app and verify each criterion

## Output File

`Console output (interactive checklist)`

## Example Usage

```
/speckit.checklist Focus on UX and accessibility requirements.
```

## Tips & Best Practices

- Great for final demo walkthrough — verify each acceptance criterion against the running app
- Can be domain-specific: 'Focus on security', 'Focus on accessibility', 'Focus on data integrity'
- Combine with /speckit.analyze for comprehensive post-build validation

# /speckit.taskstoissues

**Phase 3.5 — Project Management (Optional)**

## Purpose

Convert the generated task list from tasks.md into GitHub Issues for team tracking and execution.

## When to Use

Run AFTER /speckit.tasks when you want to track tasks as GitHub Issues.

## What It Does

- Reads tasks.md and creates a GitHub Issue for each task
- Preserves task dependencies and ordering in issue descriptions
- Links related issues together
- Requires GitHub CLI (gh) to be installed and authenticated

## Output File

`GitHub Issues in your repository`

## Example Usage

```
/speckit.taskstoissues
```

## Tips & Best Practices

- Requires GitHub CLI installed and authenticated: 'gh auth login'
- Useful for team projects where multiple developers pick up tasks
- Each issue includes the spec reference for traceability

# Quick Reference: Complete Command Cheat Sheet

Copy-paste these commands into Claude Code for the SmartInvoice demo:

**Step 1 — Foundation**

```
/speckit.constitution Create principles for SmartInvoice...
```

**Step 2 — Specify (WHAT)**

```
/speckit.specify Build SmartInvoice with 3 use cases...
```

**Step 3 — Clarify**

```
/speckit.clarify
```

**Step 4 — Plan (HOW)**

```
/speckit.plan React, dark theme, MongoDB green, 18% GST...
```

**Step 5 — Tasks**

```
/speckit.tasks
```

**Step 6 — Pre-build check**

```
/speckit.analyze
```

**Step 7 — Build**

```
/speckit.implement
```

**Step 8 — Post-build check**

```
/speckit.analyze
```

**Step 9 — Validate**

```
/speckit.checklist
```

**Key Insight:** /speckit.specify defines WHAT and WHY. /speckit.plan defines HOW. This separation means changing the tech stack doesn't touch business requirements — only the plan changes.